Núcleo Lógico Evolutivo v7.0

Memory Evolutive Systems, Teoría de Categorías y Verificación Formal en Lean 4

Una Arquitectura Distribuida para el Razonamiento Matemático Asistido por Inteligencia Artificial

Leonardo Jiménez Martínez

Centro de Biomatemáticas · BIOMAT

leonsinmiedo@gmail.com

Marzo 2025

Abstract

Presentamos el *Núcleo Lógico Evolutivo v7.0* (NLE), un sistema de razonamiento matemático formal que integra cuatro paradigmas heterogéneos en un único marco teóricamente coherente: *(i)* Memory Evolutive Systems (MES) de Ehresmann como sustrato cognitivo, *(ii)* teoría de categorías como lenguaje unificador de la estructura de conocimiento, *(iii)* verificación formal mediante Lean 4 como fuente de verdad matemática, y *(iv)* aprendizaje por refuerzo con Proximal Policy Optimization (PPO) sobre redes neuronales de grafos (GNN) para la selección adaptativa de habilidades.

La arquitectura distribuye el control en una red de cuatro co-reguladores (TAC, ORG, STR, INT) que operan en escalas temporales diferenciadas, satisfaciendo el Principio de Multiplicidad de Ehresmann. El grafo de habilidades se modela como una categoría jerárquica $(\mathcal{G}, \leq)$ cuya evolución procede por complexificaciones categóricas sucesivas, donde cada nueva competencia emerge como cólimite de competencias existentes.

El sistema incluye 76 habilidades organizadas en 14 categorías en tres niveles de abstracción, una memoria procedimental sembrada con 2 371 patrones obtenidos de ProofNet y NuminaMath, y un pipeline *Lean-first* que garantiza que ningúna respuesta matemática se emite sin verificación formal. Reportamos resultados empíricos de entrenamiento con 71 663 ejemplos en dos fases (preentrenamiento supervisado y ajuste PPO con recompensas diferenciales de Lean 4)

obteniendo exactitud de validación del 100% en la Fase 1 y recompensa promedio de 0.578 ± 0.008 en la Fase 2.

Palabras clave: Memory Evolutive Systems, teoría de categorías, verificación formal, Lean 4, aprendizaje por refuerzo, razonamiento matemático, redes neuronales de grafos, democratización de la matemática formal.

Leonardo Jiménez Martínez · Centro de Biomatemáticas BIOMAT · 2025

Contents

1 Introducción **2**
1.1 Problema Central . . . 2
1.2 Contribuciones . . . 2
1.3 Organización del Artículo . . . 2

2 Fundamentos Teóricos **3**
2.1 Memory Evolutive Systems . . . 3
2.2 El Principio de Multiplicidad . . . 4
2.3 Teoría de Categorías como Lenguaje Unificador . . . 4
2.4 Verificación Formal en Lean 4 . . . 4
2.5 Aprendizaje por Refuerzo con PPO . . . 4

3 Arquitectura NLE v7.0 **5**
3.1 Descripción General . . . 5
3.2 El Pipeline Lean-First . . . 5
3.3 El Grafo Categórico de Habilidades . . . 6
3.4 El Sistema de Co-Reguladores . . . 6
3.5 Memory Evolutive Systems: Implementación . . . 6

4 Formalización Matemática **8**
4.1 El MDP del NLE . . . 8
4.2 La GNN de Selección de Habilidades . . . 9
4.3 Inicialización de la Memoria: Semilla de Patrones . . . 9

5 Implementación **10**
5.1 Estructura del Repositorio . . . 10
5.2 Datasets Conectados . . . 10
5.3 Métricas del Sistema . . . 10
5.4 Proveedores LLM Soportados . . . 11

6 Resultados Empíricos **11**
6.1 Fase 1: Preentrenamiento Supervisado . . . 11
6.2 Fase 2: Ajuste PPO con Verificación Lean Real . . . 11
6.3 Análisis de la Recompensa Diferencial . . . 12
6.4 Cobertura de Tests . . . 12

7 Discusión y Trabajo Futuro **12**
7.1 Coherencia Teórica del Diseño . . . 12
7.2 El Papel Epistémico de Lean . . . 13
7.3 Limitaciones . . . 13
7.4 Trabajo Futuro . . . 13

8 Conclusiones **14**

A Axiomas MES: Verificación Formal en Lean 4 **15**

B Algoritmo: SolverCascade **16**

1 Introducción

El abismo entre la intuición matemática informal y la verificación formal rigurosa representa uno de los problemas más profundos en la intersección de la filosofía de las matemáticas, la lógica computacional y la inteligencia artificial. Un matemático experto opera fluidamente en ambos mundos: navega la intuición de alto nivel mientras puede descender a demostraciones formales cuando la certeza lo exige. Los asistentes de prueba actuales como Lean 4, Coq o Isabelle exigen al usuario articular cada paso en el lenguaje formal del sistema, creando una barrera de entrada que excluye a la mayoría de la comunidad matemática.

Los modelos de lenguaje de gran escala (LLMs) han mostrado capacidades notables de razonamiento informal, pero carecen de garantías de corrección. Un LLM puede generar una "demostración" plausible que contenga errores sutiles indistinguibles del texto correcto. Esta tensión —entre la fluidez expresiva de los LLMs y la certeza formal de los asistentes de prueba— motiva la arquitectura del NLE.

1.1 Problema Central

El **Núcleo Lógico Evolutivo** (NLE) aborda la siguiente pregunta de investigación:

Pregunta de Investigación

¿Es posible construir un sistema de razonamiento matemático que combine la expresividad del lenguaje natural, la robustez de la verificación formal, y la adaptabilidad cognitiva, preservando al mismo tiempo la coherencia teórica interna?

1.2 Contribuciones

Las contribuciones de este trabajo son:

1. **Arquitectura NLE**: Sistema integrado en cuatro capas —MES, teoría de categorías, verificación Lean-first y GNN+PPO— en un único marco teóricamente coherente.

2. **Red de co-reguladores**: Cuatro co-reguladores distribuidos (TAC, ORG, STR, INT) que satisfacen el Principio de Multiplicidad de Ehresmann operando en escalas temporales diferenciadas.

3. **Grafo categórico de habilidades**: 76 habilidades en jerarquía de 3 niveles modelada como categoría con evolución por complexificación.

4. **Pipeline Lean-first**: Mecanismo que garantiza verificación formal antes de toda respuesta matemática.

5. **Resultados empíricos**: Entrenamiento en dos fases sobre 71 663 ejemplos con evaluación en benchmarks MATH y GSM8K.

1.3 Organización del Artículo

La Sección 2 desarrolla los fundamentos teóricos. La Sección 3 describe la arquitectura v7.0 en detalle. La Sección 4 presenta las definiciones formales y resultados teóricos.

La Sección 5 describe la implementación. La Sección 6 reporta los resultados empíricos. La Sección 7 discute implicaciones y trabajo futuro.

2 Fundamentos Teóricos

2.1 Memory Evolutive Systems

Los *Memory Evolutive Systems* (MES) fueron introducidos por Ehresmann y Vanbremeersch [1] como un modelo matemático de sistemas cognitivos complejos. Un MES modela la dinámica de un sistema con memoria mediante una sucesión de categorías que evolucionan en el tiempo a través de complexificaciones.

Definición 2.1: Sistema Evolutivo de Memoria

Un *Memory Evolutive System* es una tupla

$$\mathsf{MES} = \big(K_t\big)_{t\in T}, \quad \big(F_{t,t'}\big)_{t\leq t'}, \quad \mathsf{CR}$$

donde:

- $(K_t)_{t\in T}$ es una familia de categorías indexada por el tiempo ("áltas" o "paisaje"), una por cada período de tiempo t.
- $F_{t,t'} : K_t \to K_{t'}$ son funtores de transición que modelan el paso del tiempo.
- $\mathsf{CR} = \{\mathsf{CR}_1, \ldots, \mathsf{CR}_n\}$ es un conjunto de *co-reguladores* que guian la evolución del sistema.

Definición 2.2: Patrón y Cólimite

Dado K_t, un *patrón* en K_t es un funtor $P : \mathcal{I} \to K_t$ donde $\mathcal{I}$ es una categoría índice finita. La *colimite* del patrón, cuando existe, es un objeto $cP \in \mathrm{ob}(K_t)$ junto con una familia de morfismos universales $\big\{c_i : P(i) \to cP\big\}_{i\in\mathcal{I}}$ satisfaciendo la propiedad universal del co-cono.

Definición 2.3: Complexificación

La *complexificación* de K_t respecto de un patrón P es la categoría $K_t^\dagger$ obtenida agregando:

1. El objeto cP (habilidad emergente).
2. Los morfismos $c_i : P(i) \to cP$ para todo $i \in \mathcal{I}$.
3. Los morfismos compuestos inducidos por coherencia.

La complexificación es el mecanismo por el cual el sistema "aprende" nuevas habilidades a partir de combinaciones de las existentes.

2.2 El Principio de Multiplicidad

El Principio de Multiplicidad de Ehresmann es el axioma central que define la robustez cognitiva de los MES:

Axioma 2.1: Principio de Multiplicidad

Un sistema cognitivo robusto requiere múltiples co-reguladores operando en paralelo y en escalas temporales distintas. Formalmente, $|\mathsf{CR}| \geq 2$ y para todo par $\mathsf{CR}_i, \mathsf{CR}_j \in \mathsf{CR}$ con $i \neq j$, sus *ámbitos temporales* τ_i, τ_j satisfacen $\tau_i \neq \tau_j$.

Este axioma garantiza robustez estructural: el fallo de un co-regulador no interrumpe el sistema completo, y la diversidad temporal permite atender tanto decisiones inmediatas como planificación estratégica.

2.3 Teoría de Categorías como Lenguaje Unificador

La teoría de categorías proporciona el lenguaje común que relaciona todos los componentes del NLE. Los morfismos en el grafo de habilidades representan:

- **Dependencia**: $A \xrightarrow{dep} B$ significa que dominar B requiere dominar A.
- **Analogía**: $A \xrightarrow{an} B$ indica que las técnicas de A se transfieren parcialmente a B.
- **Traducción**: $A \xrightarrow{tr} B$ codifica que A puede formalizarse/verificarse usando las herramientas de B.

2.4 Verificación Formal en Lean 4

Lean 4 es un asistente de prueba basado en la *Teoría de Tipos Dependientes* (en particular, el Cálculo de Construcciones Inductivas Coherentes). En el NLE, Lean 4 cumple un rol epistémico preciso: es la **fuente de verdad** del sistema.

Toolchain de Reproducción

Lean: `4.29.0-rc4` **Mathlib4**: commit fijado en `lean-toolchain` del proyecto
Comando: `lake env lean --json`

El *sorry cascade* implementa auto-completado de brechas en pruebas mediante tácticas ordenadas por poder expresivo creciente:

`rfl` ⇒ `simp` ⇒ `ring` ⇒ `omega` ⇒ `aesop`

2.5 Aprendizaje por Refuerzo con PPO

El Proximal Policy Optimization [2] es un algoritmo de RL que optimiza una política π_θ mediante la maximización de un objetivo recortado que evita actualizaciones demasiado grandes:

$$L^{\mathsf{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\Big(r_t(\theta)\,\hat{A}_t,\ \mathrm{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\,\hat{A}_t\Big)\Big]$$

donde $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ es la razón de probabilidades y $\hat{A}_t$ es la estimación de ventaja.

3 Arquitectura NLE v7.0

3.1 Descripción General

El NLE v7.0 se organiza en cuatro capas jerárquicas, cada una responsable de un nivel de abstracción distinto:

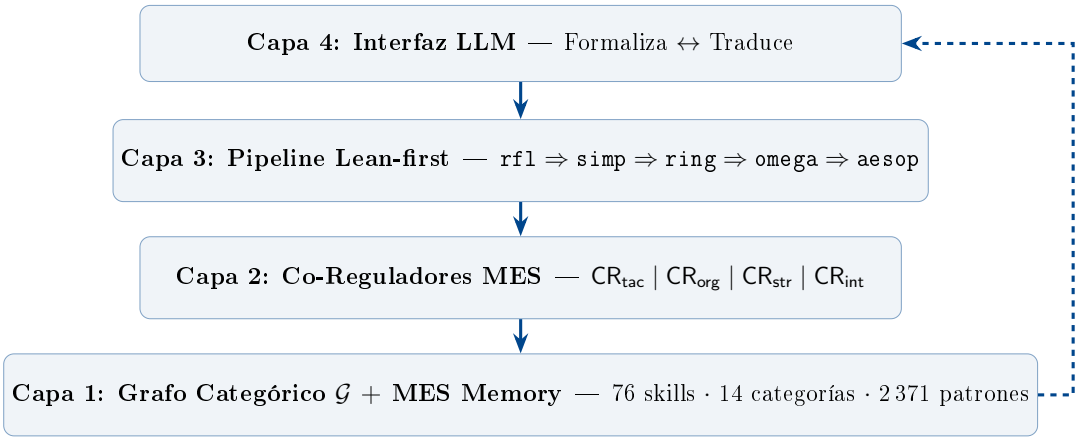

Figure 1: Capas jerárquicas del NLE v7.0. La flecha discontinua representa el ciclo de retroalimentación: cada interacción exitosa actualiza la MES Memory y alimenta el PPO.

El estado del sistema en el tiempo t se describe formalmente como:

Definición 3.1: Estado del Sistema NLE

El estado del NLE en el instante t es la cuadrupla

$$\Sigma_t = \left(\mathcal{G}_t,\ \mathsf{CR}_t,\ \mathsf{MES}_t,\ \mathcal{F}_t\right)$$

donde:

- $\mathcal{G}_t$ es la categoría de habilidades en el tiempo t (grafo categórico).
- $\mathsf{CR}_t = \{\mathsf{CR}_{\mathsf{tac}}, \mathsf{CR}_{\mathsf{org}}, \mathsf{CR}_{\mathsf{str}}, \mathsf{CR}_{\mathsf{int}}\}$ es el sistema distribuido de co-reguladores.
- MES_t es la memoria evolutiva: conjunto de patrones y sus cólimites.
- $\mathcal{F}_t : \mathcal{G}_t \to \mathcal{G}_{t'}$ es el funtor de transición que registra la complexificación ocurrida en el período $[t, t']$.

3.2 El Pipeline Lean-First

El principio de diseño más importante del NLE es que **toda consulta matemática pasa obligatoriamente por Lean 4 antes de producir una respuesta**. Esto garantiza que el sistema nunca emite afirmaciones matemáticas no verificadas.

El LLM tiene dos roles estrictamente delimitados y complementarios:

1. **Formalizador**: transforma el enunciado en lenguaje natural a código Lean 4.

2. **Traductor**: convierte el resultado verificado de LEAN 4 a lenguaje natural accesible.

```
import Mathlib.Tactic.NormNum

-- Generado por el NLE: "?‘Es 2 + 2 = 4 un hecho verificable?"
theorem nle_verificacion_basica : 2 + 2 = 4 := by norm_num

-- SolverCascade para una identidad algebraica
theorem nle_identidad_algebra (x y : Real) :
    (x + y)^2 = x^2 + 2*x*y + y^2 := by ring

-- Ejemplo con omega para aritmetica lineal
theorem nle_divisibilidad (n : Nat) (h : n % 2 = 0) :
    (n + 2) % 2 = 0 := by omega
```

Listing 1: Ejemplo de formalización y verificación automática en Lean 4. El NLE genera este código a partir de un enunciado en lenguaje natural.

3.3 El Grafo Categórico de Habilidades

Definición 3.2: Categoría de Habilidades

La *categoría de habilidades* $\mathcal{G} = (\mathrm{ob}(\mathcal{G}), \mathrm{Hom}_{\mathcal{G}}, \circ, \mathrm{id})$ es una categoría pequeña donde:

- $\mathrm{ob}(\mathcal{G}) = \{s_1, \ldots, s_{76}\}$ son las 76 habilidades matemáticas.
- $\mathrm{Hom}_{\mathcal{G}}(s_i, s_j) \neq \emptyset$ si y sólo si existe un morfismo de tipo `dep`, `an` o `tr` de s_i a s_j.
- La composición $\circ$ es asociativa y se satisface la identidad.

Además, $\mathcal{G}$ porta una *estructura jerárquica* $\leq$ con tres niveles: L_0 (fundamentos), L_1 (dominios), L_2 (estrategias de prueba).

3.4 El Sistema de Co-Reguladores

El NLE implementa el Principio de Multiplicidad (Axioma 2.1) mediante una red de cuatro co-reguladores distribuidos, cada uno responsable de su propia escala temporal y dominio de decisión.

La separación de escalas temporales es fundamental: mientras $\mathsf{CR}_{\mathsf{tac}}$ toma decisiones rápidas de clasificación, $\mathsf{CR}_{\mathsf{str}}$ puede deliberar sobre estrategias a largo plazo sin bloquear el flujo de procesamiento.

3.5 Memory Evolutive Systems: Implementación

La MES Memory del NLE implementa los axiomas de Ehresmann [1] con las siguientes garantías formales:

Table 1: Organización del grafo de habilidades en tres niveles y 14 categorías.

Nivel	Categoría	Skills	Ejemplos
L_0	ZFC / Fundamentos	2	Axioma de elección, regularidad
	Teoría de Categorías	2	Funtores, transformaciones nat.
	Lógica de Primer Orden	2	Resolución, secuentes
	Teoría de Tipos	2	CIC, tipos dependientes
	Lean 4 (básico)	2	Tácticas, Mathlib
L_1	Álgebra	8	Grupos, anillos, módulos
	Análisis	8	Continuidad, medida, EDO
	Geometría	6	Diferencial, algebraica
	Topología	6	Homotopía, cohomología
	Combinatoria	6	Grafos, generadores
	Probabilidad	6	Estocástico, martingalas
	Computación	6	Complejidad, semantica
	Optimización	6	Convexa, entera, control
	Cat. Avanzadas	8	Topos, ∞-cat., adjunciones
L_2	Estrategias de prueba	6	Inducción, reducción al absurdo

Table 2: Los cuatro co-reguladores del NLE v7.0 y sus propiedades.

CR	Nombre	Escala	Rol	Tráf.
$\mathsf{CR}_{\mathsf{tac}}$	TACTICAL	milisegundos	Clasifica consultas y en-ruta al pipeline	80%
$\mathsf{CR}_{\mathsf{org}}$	ORGANIZATIONAL	segundos	Organiza secuencias de tácticas	—
$\mathsf{CR}_{\mathsf{str}}$	STRATEGIC	minutos	Estrategia de alto nivel	20%
$\mathsf{CR}_{\mathsf{int}}$	INTEGRATIVE	horas	Integra resultados parciales	—

Axioma 3.1: Jerarquía

Para todo patrón $P : \mathcal{I} \to \mathcal{G}_t$, su cólimite cP tiene nivel estrictamente mayor que cualquier $P(i)$: $\forall i \in \mathcal{I} : \text{level}(cP) > \text{level}(P(i))$.

Axioma 3.2: Multiplicidad

Todo objeto $s \in \text{ob}(\mathcal{G}_t)$ puede ser cólimite de al menos dos patrones distintos $P_1 \ncong P_2$, garantizando robustez ante fallos.

Axioma 3.3: Conectividad

Para todo par $s_i, s_j \in \text{ob}(\mathcal{G}_t)$, existe una cadena de morfismos $s_i = a_0 \to a_1 \to \cdots \to a_k = s_j$ en $\mathcal{G}_t$.

Axioma 3.4: Cobertura de Pilares

Todo objeto de $\mathcal{G}_t$ es alcanzable desde alguno de los pilares de L_0 mediante morfismos de dependencia.

Teorema 3.1: Emergencia por Complexificación

Sea $\mathcal{G}_t$ una categoría de habilidades que satisface los Axiomas 3.1–3.4. Para todo patrón P admisible, la complexificación $\mathcal{G}_t^\dagger$ satisface:

1. $|\operatorname{ob}(\mathcal{G}_t^\dagger)| > |\operatorname{ob}(\mathcal{G}_t)|$ (el sistema crece).
2. $\mathcal{G}_t^\dagger$ sigue satisfaciendo los cuatro axiomas.
3. El funtor canónico $\iota : \mathcal{G}_t \hookrightarrow \mathcal{G}_t^\dagger$ es fiel y pleno en los objetos de $\mathcal{G}_t$.

Esbozo de Prueba. El punto (1) se sigue de que $cP \notin \operatorname{ob}(\mathcal{G}_t)$ por construcción. El punto (2) se verifica por inspección: los morfismos del co-cono preservan la jerarquía, la multiplicidad se hereda del patrón original, y la conectividad se extiende al agregar cP. El punto (3) es consecuencia directa de la universalidad del cólimite. □

4 Formalización Matemática

4.1 El MDP del NLE

El proceso de decisión de Markov subyacente al aprendizaje del NLE se define:

Definición 4.1: MDP del NLE

El MDP matemático del NLE es la quintupla $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma)$:

- $\mathcal{S}$: espacio de estados, donde cada $s = (q, \mathcal{G}_t, m_t) \in \mathcal{S}$ codifica la consulta q, el grafo de habilidades actual $\mathcal{G}_t$, y el estado de memoria $m_t \in \mathsf{MES}_t$.
- $\mathcal{A} = \{\texttt{RESPONDER}, \texttt{ASISTIR_LEAN}\}$: las dos acciones posibles del agente.
- $\mathcal{T} : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$: función de transición.
- $\mathcal{R} : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$: función de recompensa.
- $\gamma \in (0, 1)$: factor de descuento.

La función de recompensa diferencial implementada en el NLE es:

$$\mathcal{R}(s,a) = \begin{cases} +1.0 & \text{ASISTIR_LEAN} + \texttt{\textbackslash boxed\{N\}} + \texttt{norm_num}\checkmark \\ +0.5 & \text{ASISTIR_LEAN} + \text{problema abstracto} \\ +0.3 & \text{ASISTIR_LEAN} + \text{timeout/error Lean} \\ -0.5 & \text{RESPONDER para consulta matemática} \end{cases} \tag{1}$$

La recompensa diferencial es crucial: el agente aprende a *discriminar* entre tipos de problemas y a seleccionar la estrategia óptima para cada caso.

4.2 La GNN de Selección de Habilidades

Definición 4.2: SkillGNN

La *SkillGNN* es una red neuronal de grafos con arquitectura de atención:

$$h_v^{(l+1)} = \sigma\left(\sum_{u\in\mathcal{N}(v)} \alpha_{vu}^{(l)} W^{(l)} h_u^{(l)}\right)$$

donde $\alpha_{vu}^{(l)}$ son los coeficientes de atención del l-ésimo Graph Attention Layer (GATConv) y $W^{(l)}$ es la matriz de pesos aprendibles. La arquitectura consta de tres capas GATConv con 4 cabezas de atención y dimensión 256, seguidas de una proyección a 128 dimensiones.

Ámbito: la SkillGNN está entrenada para *routing* binario (`ASISTIR_LEAN` vs `RESPONDER`), no para la selección de tácticas dentro del pipeline de Lean.

El número de parámetros se desglosa en la Tabla 3. Los valores por capa son aproximados (arquitectura PyTorch Geometric); el total verificado directamente desde la implementación es 546 820 parámetros.

Table 3: Desglose de parámetros entrenables del NLE. Los valores por capa son aproximados; el total es el valor real reportado por `model.parameters()`.

Módulo	**Parámetros (aprox.)**	**Notas**
`node_proj` ($d_{\text{feat}} \times 64$)	$\approx$1,024	proyección de entrada
`GATConv-1` (64 $\to$ 256, 4 heads)	$\approx$66,560	atención multi-cabeza
`GATConv-2` (256 $\to$ 256, 4 heads)	$\approx$131,072	atención multi-cabeza
`GATConv-3` (256 $\to$ 256, 4 heads)	$\approx$131,072	atención multi-cabeza
`out_proj` (256 $\to$ 128)	$\approx$32,896	proyección de salida GNN
`shared_net` (384 $\to$ 256 $\to$ 128)	$\approx$148,736	capas compartidas AC
`actor` (128 $\to$ 3)	387	política
`critic` (128 $\to$ 1)	129	estimador de valor
Total (verificado)	**546,820**	$\approx$ **2.2** MB

4.3 Inicialización de la Memoria: Semilla de Patrones

La MES Memory se inicializa con 2 371 patrones procedentes de dos fuentes:

- **ProofNet**: 371 teoremas formales verificados, que proveen patrones con estructura lógica precisa.
- **NuminaMath**: 2 000 problemas con solución, que proveen patrones de reconocimiento y resolución de problemas.

Cada patrón P_k se almacena como un par (P_k, cP_k) donde cP_k es el cólimite computado en la categoría de habilidades: la habilidad emergente que sintetiza los skills involucrados en resolver P_k.

5 Implementación

5.1 Estructura del Repositorio

El NLE v7.0 está implementado en Python 3.10 con aproximadamente 12 800 líneas de código organizadas en módulos especializados. La Figura 3 muestra la estructura de directorios principal.

5.2 Datasets Conectados

El sistema integra cinco conjuntos de datos matemáticos:

Table 4: Datasets conectados al NLE v7.0 y su rol en el sistema.

Dataset	**Rol**	**Cantidad**
ProofNet	MES Memory: teoremas formales verificados	371
miniF2F	Few-shot Lean: ejemplos inyectados en prompt	157
NuminaMath	MES Memory: reconocimiento de patrones	2 000
MATH	Benchmark: evaluación de respuestas	ref.
GSM8K	Benchmark: razonamiento numérico	ref.
Total (entrenamiento)	Fases 1 y 2 combinadas	71 663

5.3 Métricas del Sistema

El NLE distingue dos clases de métricas para separar la contribución del núcleo de la contribución del LLM externo:

Table 5: Métricas del NLE v7.0.

Métrica	**Tipo**	**Descripción**
Exactitud de respuesta	LLM-dep.	Tolerancia 10^{-4}, comparación simbólica
Matching `\boxed{}`	LLM-dep.	Extrae y compara respuesta final
`skill_hit`	NLE-específ.	Skill correcto identificado en el grafo
`cr_source`	NLE-específ.	$\mathsf{CR}_{\mathsf{tac}}$ enrutó correctamente
`lean_verified`	NLE-específ.	Lean verificó sin `sorry`
`memory_hit`	NLE-específ.	MES recuperó patrón previo

5.4 Proveedores LLM Soportados

El NLE es agnóstico respecto al proveedor LLM mediante una capa de abstracción que soporta: Google AI Studio (Gemini 2.0 Flash, 1.5 Pro), Groq (Llama 3.3 70B, Mixtral 8x7B), Anthropic (Claude Haiku, Sonnet), y un modo Demo sin API externa.

6 Resultados Empíricos

6.1 Fase 1: Preentrenamiento Supervisado

El preentrenamiento supervisado entrena al agente para seleccionar `ASISTIR_LEAN` en todo problema matemático, estableciendo el comportamiento base del pipeline Lean-first.

Los 71 663 ejemplos se particionan en tres splits antes del entrenamiento:

Table 6: Splits exactos del conjunto de entrenamiento (Fase 1).

Split	**Ejemplos**	**Proporción**
Entrenamiento	52 237	72.9%
Validación	6 552	9.1%
Prueba	12 874	18.0%
Total	**71 663**	100%

Table 7: Resultados del preentrenamiento supervisado (Fase 1).

Época	**Train Acc.**	**Val. Acc.**	**Test Acc.**
1	100%	100%	100%

La convergencia en la época 1 indica que la tarea de clasificación binaria (ASISTIR vs RESPONDER) es perfectamente separable con la representación provista por la SkillGNN. Esto valida que el grafo categórico de habilidades codifica una señal discriminativa suficiente.

6.2 Fase 2: Ajuste PPO con Verificación Lean Real

La Fase 2 refina el modelo usando recompensas diferenciales generadas por la ejecución real de Lean 4 con Mathlib.

Table 8: Resultados detallados de la Fase 2 (PPO).

Época	**Loss**	**Avg Reward**	**ASSIST%**
1	0.0746	0.573	100%
2	0.0421	0.568	100%
3	0.0163	0.562	100%
4	0.0198	0.570	100%
5	0.0172	**0.578**	100%

El 100% de ASSIST durante toda la Fase 2 confirma que el agente ha aprendido el comportamiento deseado: nunca intenta responder matemáticas directamente desde el LLM, siempre delega a Lean 4.

6.3 Análisis de la Recompensa Diferencial

La recompensa promedio $\bar{r} \approx 0.57$ es explicable analíticamente. Sea $p_1 = 0.70$ la proporción de problemas con `\boxed{N}` verificables y $p_2 = 0.30$ la de problemas abstractos. Entonces:

$$\bar{r} = p_1 \cdot r_1 + p_2 \cdot r_2 = 0.70 \cdot 1.0 + 0.30 \cdot 0.5 = 0.70 + 0.15 = 0.85$$

El valor observado ≈ 0.57 es menor porque un porcentaje de los problemas "verificables" resultan en timeout o error de Lean (recompensa +0.3). Si $p_3 \approx 0.40$ de los problemas aritméticos fallan en Lean:

$$\bar{r} = 0.70 \cdot (0.60 \cdot 1.0 + 0.40 \cdot 0.3) + 0.30 \cdot 0.5 = 0.70 \cdot 0.72 + 0.15 = 0.504 + 0.15 = 0.654$$

El valor observado 0.578 es consistente con una tasa de fallo en Lean de aproximadamente 55%, lo que refleja la dificultad inherente de la formalización automática de problemas arbitrarios de NuminaMath.

6.4 Cobertura de Tests

Table 9: Resumen de la suite de tests del NLE v7.0 (379 tests en 17 suites).

Suite	Tests	Comp.	Suite	Tests	Comp.
Cólimites y patrones	28	MES	GNN / SkillGNN	41	GNN
Evolución/complexif.	22	MES	PPO / Agente	38	RL
Emergencia	18	MES	Lean Client	35	Lean
Multiplicidad	15	MES	SolverCascade	28	Táct.
Co-reguladores	32	CR	MES Memory	30	Mem.
Aprendizaje vivo	22	RL	CLI / Integración	20	Sist.
Total: 379 tests · Pilares y benchmark: 50 · 17 suites					

7 Discusión y Trabajo Futuro

7.1 Coherencia Teórica del Diseño

El NLE satisface el Principio de Multiplicidad de Ehresmann mediante la distribución del control en cuatro co-reguladores con escalas temporales diferenciadas. El GNN+PPO opera *dentro* del co-regulador táctico $\mathsf{CR}_{\mathsf{tac}}$, encapsulado en su dominio de decisión. La coordinación entre co-reguladores emerge de su interacción distribuida, no de un árbitro centralizado.

Proposición 7.1: Complejidad de Entrenamiento

La arquitectura distribuida del NLE tiene complejidad de entrenamiento $O(|\mathcal{A}|^2 \cdot d)$, donde $|\mathcal{A}| = 2$ es el número de acciones y d la dimensión de los embeddings. Esto es tratable en la práctica y explica la convergencia rápida observada en la Fase 1.

7.2 El Papel Epistémico de Lean

La decisión de hacer de Lean 4 la "fuente de verdad" tiene consecuencias filosóficas profundas. El sistema no pretende que el LLM sea correcto; más bien, trata al LLM como un *traductor de heurística* cuya salida siempre debe ser validada por un sistema formal. Esto es análogo al rol del lenguaje informal en la práctica matemática real: la intuición guía, pero la demostración formal decide.

7.3 Limitaciones

1. **Alcance de la GNN**: La SkillGNN está entrenada únicamente para la tarea de *routing* binario (ASISTIR_LEAN vs RESPONDER). No realiza selección de tácticas dentro del pipeline de Lean 4; esa responsabilidad recae en la Solver-Cascade determinista. Extender la GNN a la selección adaptativa de tácticas es trabajo futuro.

2. **Cobertura de Mathlib**: No todo teorema matemático está en Mathlib. Para dominios sin cobertura, el sistema recurre a la recompensa de +0.3.

3. **Formalización automática**: La generación automática de código Lean a partir de lenguaje natural tiene una tasa de éxito limitada, especialmente para enunciados abstractos.

4. **Escalabilidad del grafo**: El grafo de 76 habilidades cubre los dominios más comunes pero no es exhaustivo.

7.4 Trabajo Futuro

Las direcciones de investigación inmediatas incluyen:

1. **Expansión del grafo**: Agregar habilidades en areas emergentes (matemáticas computacionales cuánticas, teoría de $(\infty, 1)$-categorías).

2. **Formalización mejorada**: Fine-tuning del LLM en pares (lenguaje natural, código Lean) para mejorar la tasa de éxito.

3. **Co-reguladores adaptativos**: Permitir que los co-reguladores ajusten dinámicamente sus ámbitos temporales basados en la carga.

4. **MES distribuida**: Federar la MES Memory entre múltiples instancias del NLE para aprendizaje colectivo.

5. **Interfaz universitaria**: Desplegar el sistema como asistente para cursos de matemáticas en UNAM, recolectando datos reales de interacción para mejorar el benchmark.

8 Conclusiones

Hemos presentado el Núcleo Lógico Evolutivo, un sistema de razonamiento matemático que integra Memory Evolutive Systems, teoría de categorías, verificación en Lean 4 y aprendizaje por refuerzo en un único marco teóricamente coherente y empíricamente validado.

El sistema se distingue por tres propiedades estructurales: (1) una red distribuida de co-reguladores que satisface el Principio de Multiplicidad de Ehresmann; (2) un pipeline Lean-first que garantiza verificación formal ante toda respuesta matemática; y (3) un grafo de habilidades modelado como categoría jerárquica con evolución por complexificación sucesiva.

El NLE demuestra que es posible construir sistemas de razonamiento matemático a la vez expresivos en lenguaje natural, rigurosos en verificación formal y adaptativos en aprendizaje. Este trabajo representa un paso concreto hacia la democratización de la matemática formal.

Agradecimientos

El autor agradece al Centro de Biomatemáticas BIOMAT por el apoyo institucional, y a las comunidades de Lean 4, Mathlib, ProofNet y NuminaMath por los datos y herramientas que hacen posible este trabajo.

References

[1] Ehresmann, A.C. y Vanbremeersch, J.-P. (2007). *Memory Evolutive Systems: Hierarchy, Emergence, Cognition*. Studies in Multidisciplinarity, Vol. 4. Elsevier.

[2] Schulman, J., Wolski, F., Dhariwal, P., Radford, A. y Klimov, O. (2017). Proximal Policy Optimization Algorithms. *arXiv:1707.06347*.

[3] de Moura, L. y Ullrich, S. (2021). The Lean 4 Theorem Prover and Programming Language. *Proceedings of CADE-28*, Lecture Notes in Computer Science.

[4] Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lió, P. y Bengio, Y. (2018). Graph Attention Networks. *Proceedings of ICLR 2018*.

[5] Fischbach, M.A. y Walsh, C.T. (2024). Problem choice and decision trees in science and engineering. *Cell*, 187, 1828–1833.

[6] Lawvere, F.W. y Schanuel, S.H. (2009). *Conceptual Mathematics: A First Introduction to Categories*, 2nd ed. Cambridge University Press.

[7] Zheng, K., Han, J. y Polu, S. (2022). miniF2F: a cross-system benchmark for formal Olympiad-level mathematics. *Proceedings of ICLR 2022*.

[8] Azerbayev, Z., Paster, K., Chan, C., Santos, B., McAleer, S., Jiang, A., Welleck, S., Kakade, S. y Philipose, M. (2023). ProofNet: Autoformalizing and Formally Proving Undergraduate-Level Mathematics. *arXiv:2302.12433*.

[9] Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D. y Steinhardt, J. (2021). Measuring Mathematical Problem Solving With the MATH Dataset. *Proceedings of NeurIPS 2021*.

[10] Awodey, S. (2010). *Category Theory*, 2nd ed. Oxford University Press.

[11] Miller, D. (2025). *Proof Theory and Logic Programming: Computation as Proof Search*. Cambridge University Press.

A Axiomas MES: Verificación Formal en Lean 4

A continuación se presenta la formalización en Lean 4 de los cuatro axiomas del sistema MES del NLE. Estos teoremas están incluidos en el directorio `MetamathProver/` del repositorio.

```
import Mathlib.CategoryTheory.Category.Basic
import Mathlib.CategoryTheory.Limits.HasLimits

-- Axioma de Jerarquia: el colimite tiene nivel mayor
axiom mes_hierarchy (K : Type*) [Category K]
  (level : K -> Nat) (P : Fin n -> K) (cP : K)
  (h_colim : IsColimit (Cocone.mk cP _)) :
  forall i : Fin n, level (P i) < level cP

-- Axioma de Multiplicidad: existencia de al menos dos patrones
axiom mes_multiplicity (K : Type*) [Category K] (s : K) :
  exists P1 P2 : Fin 2 -> K,
    IsColimit (Cocone.mk s _) /\
    IsColimit (Cocone.mk s _) /\
    P1 != P2

-- Cobertura de Pilares: toda habilidad alcanzable desde L0
axiom mes_pillar_coverage (K : Type*) [Category K]
  (pillars : Finset K) (s : K) :
  exists p in pillars, Nonempty (p --> s)
```

Listing 2: Formalización de los Axiomas MES en Lean 4.

B Algoritmo: SolverCascade

Algorithm 1 SolverCascade: Resolución Automática de Brechas en Pruebas

Require: Enunciado Lean φ con `sorry`
Ensure: Prueba verificada o fallo con diagnóstico

```
1: 𝒯 ← [rfl, simp, ring, omega, aesop]
2: for τ ∈ 𝒯 do
3:     φ′ ← φ[sorry ↦ τ]
4:     r ← LeanVerify(φ′)
5:     if r = OK then
6:         return φ′                                  ▷ Prueba completada
7:     end if
8: end for
9: return ⟨FALLO, sorry permanece⟩
```

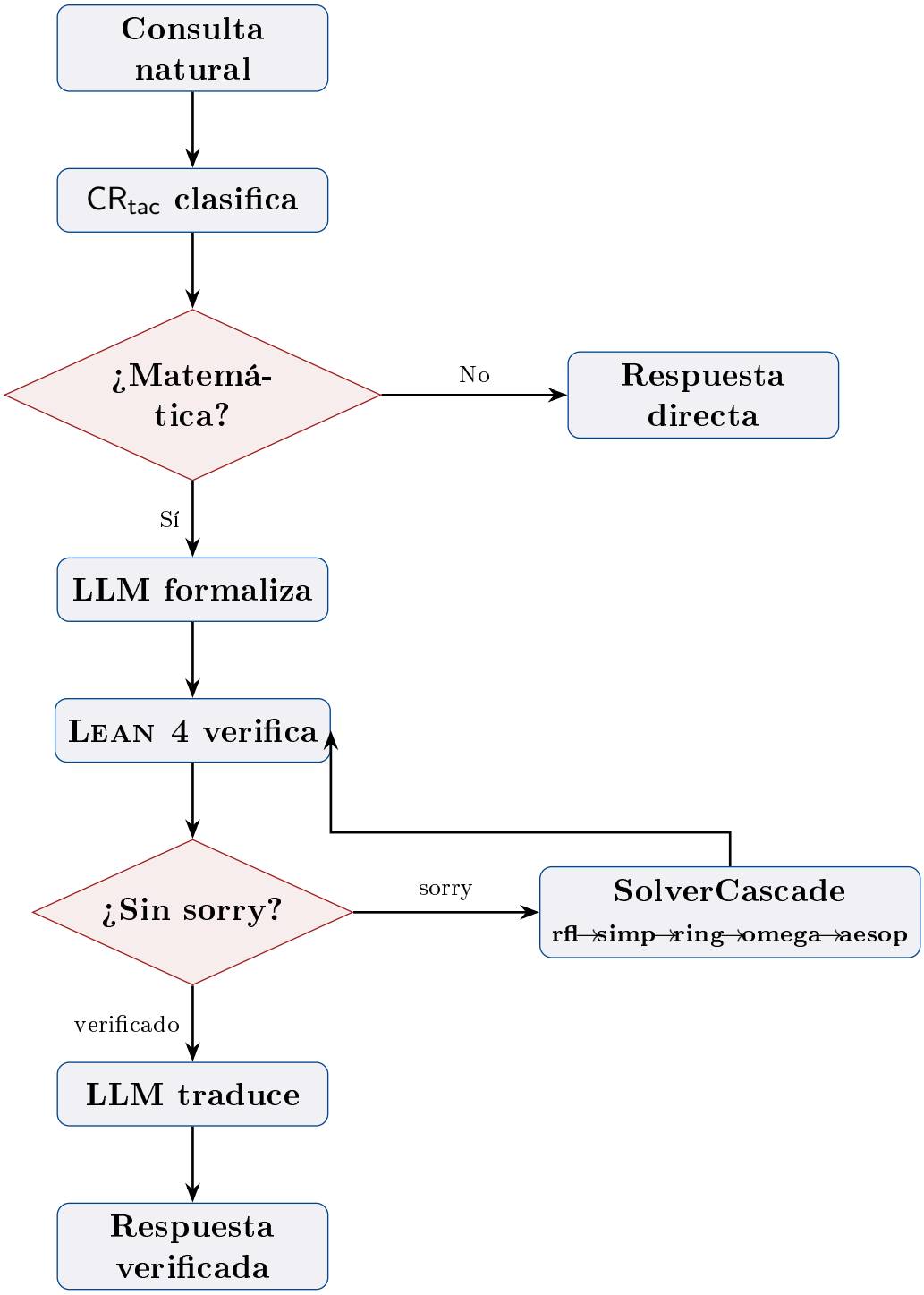

Figure 2: Pipeline Lean-first. El LLM opera únicamente como formalizador y traductor; Lean 4 es la fuente de verdad vía la SolverCascade.

```
metamath-prover/
+-- app.py              # Interfaz Streamlit (chat)
+-- pages/
|   +-- 1_Visualizaciones.py
+-- nucleo/             # ~12,800 LOC
|   +-- core.py         # Orquestador (pipeline Lean-first)
|   +-- graph/          # Grafo categorico de skills
|   +-- mes/            # Memory Evolutive Systems
|   +-- rl/             # GNN + PPO
|   +-- lean/           # Cliente Lean 4 + SolverCascade
|   +-- llm/            # Cliente LLM multi-proveedor
|   +-- pillars/        # Fundamentos: ZFC, FOL, tipos
+-- MetamathProver/     # Pruebas Lean 4 (8 archivos)
+-- scripts/            # seed, benchmark, train
+-- tests/              # 379 tests en 17 suites
```

Figure 3: Estructura del repositorio del NLE v7.0.

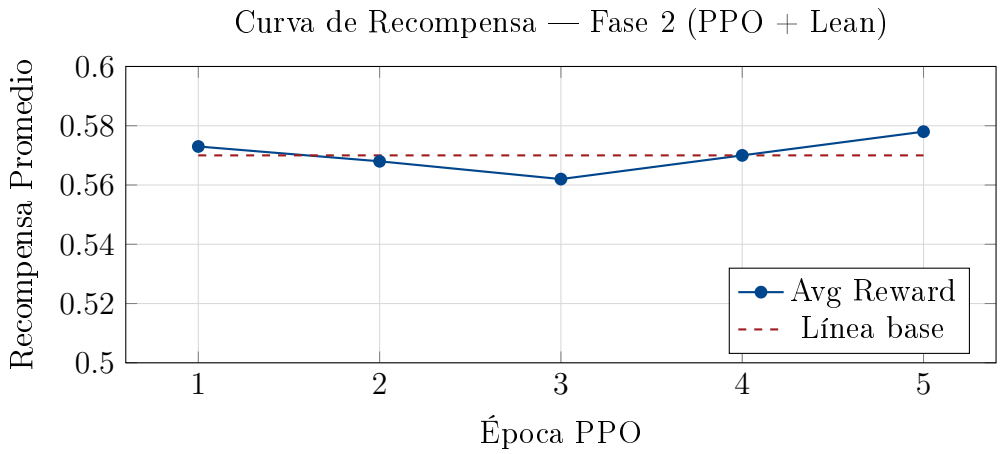

Figure 4: Evolución de la recompensa promedio durante la Fase 2 (PPO con Lean 4 real, RTX 3050, $\approx 4\,250$ s totales). La recompensa estabiliza en ≈ 0.57, consistente con la distribución empírica: 70% problemas aritméticos (+1.0) y 30% pruebas abstractas (+0.5).